

SELF-STUDY GUIDE

Databricks Free Edition

Taking the course on your own, step by step

Contents

PART 1	Before you start	3
PART 2	Set up the workspace	4
PART 3	Working with the notebooks	6
PART 4	The course step by step	7
PART 5	Jobs in Free Edition	10
PART 6	Differences from the instructor-led course	12
PART 7	Troubleshooting	13
PART 8	Cleaning up	13

This guide shows how to take the Databricks Fundamental course on your own in Databricks Free Edition. Free Edition costs nothing and needs no cloud subscription. Every module, both workshops and the final project run there without changes: the setup notebooks recognise a single-user workspace and adjust automatically.

Item	Detail
What you need	A Databricks Free Edition account and a web browser
Setup time	About 10 minutes
Course time	About 7 hours, or longer if you also do the BONUS sections
Compute	Serverless only – you never create a cluster
Course repository	https://github.com/ditw-training/databricks-fundamental

PART 1

Before you start

What Free Edition is

Databricks Free Edition is a free, non-commercial workspace for learning. It contains the same platform you would use at work: notebooks, Unity Catalog, Delta Lake, Lakeflow Jobs, the SQL Editor and AI/BI. You sign in with an e-mail one-time code or a Google or Microsoft account.

Limits that matter for this course

Limit	What it means for you
Serverless compute only	Notebooks and jobs always run on Serverless. A few demos that need a classic cluster are skipped automatically (see Part 6)
One SQL warehouse, size 2X-Small	Enough for the SQL Editor, dashboards and Genie in this course
At most 5 job tasks running at the same time	The course jobs have three tasks each – run one job at a time
One workspace, no account console	You cannot create account groups; the Unity Catalog demo uses a built-in group instead
No custom storage locations	Your catalog is stored in the workspace's built-in storage
Daily usage quota	If you exceed it, compute is switched off until the next day. Your data, notebooks and settings are kept
Non-commercial use only	Use the workspace for learning, not for company data

Databricks can change these limits; the current list is in the Databricks documentation under *Free Edition limitations*.

How the course is organised

The repository contains three kinds of notebooks:

Folder	Content	How to use it
notebooks/setup	Environment setup	Run <code>00_pre_config</code> once, then <code>00_setup</code> once
notebooks/modules	Demo modules M00 to M07	Read the explanations and run the cells one by one
notebooks/ workshops	Hands-on exercises and the final project	Write your own code, then run the CHECK cell after each task

The data comes from the fictional online store RetailHub: customers, orders and products in CSV, JSON, Parquet and Excel files in the `dataset` folder.

PART 2

Set up the workspace

Step 1 – Create a Free Edition account

1. Open <https://www.databricks.com/learn/free-edition> and choose **Sign up**.
2. Sign in with your e-mail address, Google or Microsoft account.
3. Wait until the workspace opens. It already contains the catalogs `workspace`, `samples` and `system`.

The e-mail address you use decides the name of your course catalog (Step 3), so use the same account for the whole course.

Step 2 – Clone the course repository as a Git folder

1. In the left menu open **Workspace**, then **Home** (your user folder).
2. Select **Create**, then **Git folder**.
3. Enter the Git repository URL <https://github.com/ditw-training/databricks-fundamental>. The provider GitHub is detected automatically. The repository is public, so no credentials are needed.
4. Select **Create Git folder**.

The folder `databricks-fundamental` appears in your Home folder with the subfolders `notebooks`, `dataset`, `materials`, `assets` and `docs`.

Keep the notebooks inside the Git folder. The notebooks find the setup notebook and the data files through paths relative to the repository; a notebook copied elsewhere fails at `%run ../setup/00_setup`.

Step 3 – Run `00_pre_config`

1. Open `notebooks/setup/00_pre_config`.
2. In the top-right corner select **Connect**, then **Serverless**.
3. Select **Run all**.

The notebook is written for a trainer who prepares catalogs for a whole group. In Free Edition it switches to *single-user mode* and prints the following messages. They are expected, not errors.

Message	Meaning
Group 'alt_trn_gr' not found – single-user mode	There is no training group, so the environment is created only for you
STORAGE_LOCATION not usable – using the metastore default storage	The catalog is stored in the built-in storage of Free Edition
retailhub_analysts does not exist – M06 will grant to account users	The Unity Catalog module will use the built-in account users group

The last cell must print **ALL ENVIRONMENTS READY**. The notebook has created:

Object	Name
Catalog	retailhub_<name>
Schemas	bronze, silver, gold, default
Volume	retailhub_<name>.default.datasets, with a copy of the dataset folder
Permissions	ALL PRIVILEGES and MANAGE on the catalog for your account

How your catalog name is built

<name> comes from your e-mail address:

1. Take the part before @.
2. Replace every character that is not a letter or a digit with `_`, and change the letters to lower case.
3. Remove digits and `_` from the start.
4. Add the prefix `retailhub_`.

E-mail	Catalog
jan.kowalski@example.com	retailhub_jan_kowalski
anna-maria@example.com	retailhub_anna_maria
dev@example.com	retailhub_dev

You need this name later for the job parameters and the SQL Editor. `00_setup` prints it for you.

Running `00_pre_config` again is safe: it creates only what is missing and copies the data files again.

Step 4 – Run `00_setup`

Open notebooks/setup/`00_setup`, connect to Serverless and select **Run all**. The notebook checks the catalog, the schemas and the volume, and ends with:

```

TRAINING ENVIRONMENT READY
CATALOG:          retailhub_<name>
DATASET_PATH:     /Volumes/retailhub_<name>/default/datasets
IS_SERVERLESS:    True

```

Every module and workshop runs this notebook with `%run ../setup/00_setup` and receives the variables `CATALOG`, `BRONZE_SCHEMA`, `SILVER_SCHEMA`, `GOLD_SCHEMA`, `DATASET_PATH` and `IS_SERVERLESS`. You do not run it by hand again.

Step 5 – Check the environment

Open `notebooks/modules/00_intro` and run the environment verification cell. It must report your catalog with the schemas `bronze`, `default`, `gold` and `silver`. On Serverless the runtime is shown as `client.x.y`; this is correct.

PART 3

Working with the notebooks

Compute

Always connect notebooks to **Serverless**. It starts in a few seconds and stops by itself when you are idle, so there is nothing to switch off. For the SQL Editor, dashboards and Genie use the **Serverless Starter Warehouse**.

Running cells

Run the module cells one at a time and read the explanation above each cell; the course is written that way. **Run all** works too, but you learn less from it. You can re-run a module at any time: the demo cells recreate their tables.

Markers used in the notebooks

Marker	Meaning	What to do
⊕ BONUS	Outside the one-day schedule	Skip on the first pass, return to it later
[UI DEMO]	A step done in the Databricks interface, not in code	Follow the checklist in the cell
<code>%skip</code> at the top of a cell	The cell is not run by Run all	Remove the line when the text tells you to
<code># YOUR CODE HERE,</code> <code>_____, ...</code>	A place for your code in a workshop	Replace it with your solution
CHECK cell	Automatic test of the previous task	Run it; all assertions must pass before you continue

Workshops

Each workshop task has a task cell and a CHECK cell. Write the code, run the task cell, then run the CHECK cell. A failed assertion tells you what is wrong, for example `customers_df not defined` – replace the `...` placeholder with your code. Fix the task cell and run both cells again.

Optional tasks start with `%skip`. To do one, delete the `%skip` line in the task cell and in its CHECK cell.

The workshops write to their own tables (`lab_ingestion_customers`, `lab_delta_cust_ws`, `lab_orders`, `lab_daily_orders`), so they never overwrite the demo tables.

Getting updates

When the course repository is updated, open the Git dialog (select **Git** next to the folder name, or the branch button `main` in a notebook) and select **Pull**.

If Pull reports a conflict, you changed a notebook that was also updated in the repository. Copy your version outside the Git folder (**Clone** in the notebook menu), then in the Git dialog select `:`, **Reset** to discard local changes, and **Pull** again.

PART 4

The course step by step

The order below follows the one-day schedule. Times are those of the instructor-led day; working alone usually takes longer.

M00 – Intro and setup (15 min)

`notebooks/modules/00_intro`. The agenda, the RetailHub scenario and the environment check from Part 2, Step 5.

M01 – Platform and workspace (60 min)

`notebooks/modules/01_platform_workspace`. The workspace, Catalog Explorer, Git folders, compute types, costs, magic commands, Genie Code and first visualisations.

Things to do by hand:

- **Upload files to a volume.** The cell creates the volume `default.landing`. Download `stores.csv` and `promotions.json` from the repository folder `materials/volume_upload` on GitHub, then in **Catalog** open your catalog, `default`, `landing` and select **Upload to this volume**. Run the next cells to read the files.
- **Visualisations and data profile.** After a `display()` result select `+`, then **Visualization** or **Data Profile**.

Free Edition notes: the DBU reports use `system.billing.usage`, which Free Edition does not have, so those cells print that they were skipped. The Photon status is shown as “Serverless: Photon is always on”.

M02 – Spark architecture (25 min)

`notebooks/modules/02_spark_architecture`. Driver and executors, lazy evaluation, DAGs, stages and shuffles, uses the `samples.nyctaxi.trips` table.

Free Edition notes: Serverless does not let you read Spark settings, so the AQE cell shows Spark defaults marked “not readable here”. The BONUS section *Cache and Persist* is skipped because `cache()` and `persist()` are not supported on Serverless. The Spark UI of a classic cluster is not available; use the query profile shown under each cell result instead.

M03 – ELT and ingestion (45 min)

`notebooks/modules/03_medallion_ingestion`. The medallion architecture, reading CSV, JSON, Parquet and Excel, `read_files()`, transformations, filters, aggregations, joins and temporary views.

Free Edition notes: the BONUS *Global Temporary Views* is skipped — Serverless does not support global temporary views. Regular temporary views work.

Workshop – Ingestion (40 min)

`notebooks/workshops/WORKSHOP_ingestion`. Twelve tasks: imports, reading CSV with inferred and explicit schemas, exploration, JSON with a schema, filters, new columns, conditional logic, aggregations, temporary views, writing a Bronze Delta table and a BONUS join. Everything runs on Free Edition.

M04 – Delta Lake (40 min)

`notebooks/modules/04_delta_fundamentals`. Schema enforcement and evolution, INSERT, UPDATE, DELETE, MERGE, table history, time travel, RESTORE, VACUUM and managed versus external tables.

Free Edition notes: the VACUUM demo lowers `delta.deletedFileRetentionDuration` on the demo table so that `RETAIN 0 HOURS` is accepted; after VACUUM the old versions are reported as “NOT readable”, which is the point of the demo. The BONUS *External Tables* reports that no external location is available — this is expected in Free Edition.

Workshop – Delta Lake (25 min)

`notebooks/workshops/WORKSHOP_delta`. Create a Delta table, see schema enforcement reject a bad write, add a column, run INSERT, UPDATE, DELETE and MERGE, read the history, travel to version 0 and recover the table with RESTORE. Tasks 04 (constraints) and 11 (time travel by timestamp) are optional. Everything runs on Free Edition.

M05 – Orchestration and Lakeflow (35 min)

`notebooks/modules/05_orchestration_jobs`. Lakeflow Jobs: task types, a multi-task job with parameters and task values, triggers, retries, notifications, the SQL Editor and job monitoring with system tables. Build the job described in Part 5.

M06 – Unity Catalog (20 min)

`notebooks/modules/06_unity_catalog`. The three-level namespace, account groups and GRANT / REVOKE.

Free Edition notes: there is no account console, so the group `retailhub_analysts` does not exist. The GRANT cell prints that it uses the built-in group account `users` instead, and the rest of the demo — `SHOW GRANTS`, then `REVOKE SELECT` — works the same way. In Free Edition you are the only member of account `users`.

Final project (50 min)

`notebooks/workshops/final_labs/WORKSHOP_final_project`. Build the orders pipeline from raw JSON to a daily revenue table and run it as a scheduled job.



1. Run `tasks/lab_task_01_bronze` — it is pre-filled.
2. Complete and run `tasks/lab_task_02_silver` and `tasks/lab_task_03_gold`.
3. Run the smoke test (Step 2 of the project notebook).
4. Create the job described in Part 5.
5. Check the Gold table in the SQL Editor (Step 4 of the project notebook).

Expected results:

Table	Rows
<code>bronze.lab_orders</code>	100,000
<code>silver.lab_orders</code>	90,904
<code>gold.lab_daily_orders</code>	32 (one row per day, 2024-12-31 to 2025-01-31)

M07 – AI/BI dashboards and Genie (optional)

`notebooks/modules/07_aibi_dashboards`. Run the notebook cells first: they create the demo tables in your Gold schema. Then follow the steps in the notebook in the interface: queries in the SQL Editor, an AI/BI dashboard and a Genie space. Use the Serverless Starter Warehouse.

PART 5

Jobs in Free Edition

Jobs are created in **Jobs & Pipelines**, then **Create, Job**. In Free Edition the only compute for a job task is **Serverless**. Wherever the notebooks say *Shared training cluster* or *Job cluster*, choose Serverless.

The M05 demo job

The three task notebooks are ready in `materials/orchestration`.

Task name	Notebook	Depends on	Parameters
validate	materials/orchestration/task_01_validate	–	min_rows = 500
transform	materials/orchestration/task_02_transform	validate	–
report	materials/orchestration/task_03_report	transform	–

1. Create the job and name it, for example `Demo_ETL_Pipeline_<name>`.
2. Add the three tasks from the table. For each: type **Notebook**, source **Workspace**, compute **Serverless**.
3. Add the job parameter `source_table = samples.nyctaxi.trips`.
4. Add a trigger: **Scheduled, Advanced**, cron `0 0 6 * * ?`, timezone `Europe/Warsaw`, status **Paused**.
5. Select **Run now**, open the run and look at the task graph and the output of each task.

The task names matter. `transform` reads a task value from the task named `validate`, and `report` reads from `transform`. If you name the tasks differently, add the task parameter `validate_task` (in `transform`) or `transform_task` (in `report`) with the name you used. Otherwise the job still succeeds, but prints a warning and shows `-1` instead of the values.

The final project job

Task name	Notebook	Depends on
bronze_load	notebooks/workshops/final_labs/tasks/lab_task_01_bronze	–
silver_transform	notebooks/workshops/final_labs/tasks/lab_task_02_silver	bronze_load
gold_aggregation	notebooks/workshops/final_labs/tasks/lab_task_03_gold	silver_transform

Job parameters:

Parameter	Value
catalog	retailhub_<name>
source_path	/Volumes/retailhub_<name>/default/datasets

The screenshot shows a Databricks Jobs workflow with three tasks: `bronze_load`, `silver_transform`, and `gold_aggregation`. The `gold_aggregation` task is selected, and its configuration is shown in a form below.

Task configuration for `gold_aggregation`:

- Task name***: `gold_aggregation`
- Type***: Notebook
- Source***: Workspace
- Path***: `...tal/notebooks/workshops/final_labs/tasks/lab_task_03_gold`
- Compute***: Serverless (Autoscaling)
- Depends on**: `silver_transform`
- Run if dependencies**: All succeeded

Add the same paused schedule as for the M05 job and select **Run now**. Each task shows its result in the **Output** tab, for example `{"status": "SUCCESS", "table": "retailhub_<name>.gold.lab_daily_orders", "rows": 32}`. If a task fails, fix the notebook and use **Repair run**: only the failed task and the tasks after it run again.

Checking results in the SQL Editor

Open **SQL Editor**, select the **Serverless Starter Warehouse** and run:

```
SELECT * FROM retailhub_<name>.gold.lab_daily_orders ORDER BY order_date;

DESCRIBE HISTORY retailhub_<name>.gold.lab_daily_orders;
```

Keeping within the limits

- Run one job at a time: Free Edition allows five job tasks at once.
- Leave schedules **Paused**. A daily schedule keeps using your quota after you finish the course.

PART 6

Differences from the instructor-led course

The instructor-led course runs on Azure Databricks with a shared classic cluster. The notebooks detect Serverless and handle the differences; the affected cells print an explanation instead of failing.

Module	Cell	In Free Edition
M01	DBU reports	<code>system.billing.usage</code> is not available; the reports are skipped
M01	Runtime and Photon	Runtime <code>client.x.y</code> ; Photon is always on
M02	AQE settings	Spark settings cannot be read; defaults are shown
M02	BONUS Cache and Persist	Not supported on Serverless; skipped
M02	BONUS Spark UI	No Spark UI; use the query profile of a cell
M03	BONUS Global Temporary Views	Not supported on Serverless; skipped
M04	BONUS VACUUM	Works; the demo lowers the retention property of the demo table
M04	BONUS External Tables	No external location; the cell reports it
M06	GRANT / REVOKE	Uses account <code>users</code> instead of <code>retailhub_analysts</code>
M05, Final project	Job compute	Serverless instead of a cluster

Everything else — reading files from volumes, Delta Lake, MERGE, time travel, RESTORE, jobs with parameters and task values, system tables for job runs, the SQL Editor — works the same way.

PART 7

Troubleshooting

Symptom	Cause	Fix
Catalog 'retailhub_<name>' not found	00_pre_config has not been run, or you use a different account	Run 00_pre_config with the account you use for the course
%run ../setup/00_setup fails with <i>notebook not found</i>	The notebook was copied outside the Git folder	Open the notebook from databricks-fundamental
The volume datasets is empty	The data copy in 00_pre_config did not finish	Run 00_pre_config again
CONFIG_NOT_AVAILABLE or NOT_SUPPORTED_WITH_SERVERLESS	An older copy of the repository	Pull the latest version (Part 3, <i>Getting updates</i>)
A CHECK cell fails	The task is not finished or a variable has a different name	Read the assertion message, fix the task cell, run both cells again
TABLE_OR_VIEW_NOT_FOUND for lab_orders in Silver or Gold	The previous layer has not been built	Run the Bronze, then the Silver task first
The M05 job prints WARNING: no task 'validate'	The tasks have other names	Rename the tasks or set <code>validate_task</code> / <code>transform_task</code> (Part 5)
Compute is unavailable for the rest of the day	The daily usage quota was exceeded	Wait until the next day; data and notebooks are kept
Pull reports a conflict	You edited a notebook that changed in the repository	Clone your notebook elsewhere, then Reset and Pull

PART 8

Cleaning up

When you have finished the course, remove what it created. To start again later, run 00_pre_config and 00_setup again.

1. In **Jobs & Pipelines** delete the M05 job and the final project job, or at least leave their schedules paused.
2. Delete the dashboard and the Genie space from M07, if you created them.
3. Drop the course catalog in a notebook or in the SQL Editor:

```
DROP CATALOG IF EXISTS retailhub_<name> CASCADE;
```

4. Optionally delete the Git folder databricks-fundamental.